

AIOPS PLATFORM · ENGINEERING GUIDE

Deploying Dagster on Amazon EKS

Helm · Kubernetes · Run Execution · Keycloak SSO

A hands-on guide written for engineers new to Kubernetes and Dagster. It explains how the pieces fit together, why each choice is made, and the exact order to deploy them — from an empty cluster to a secured, multi-domain Dagster running behind single sign-on.

Self-hosted Dagster OSS · multi-code-location · production target
Internal teaching document — adapt values to your environment before use.

Contents

- 1 The big picture — what we are building** — and why it looks this way
- 2 Foundations** — Kubernetes / EKS, Helm, and Dagster in plain language
- 3 Dagster's architecture** — two Helm charts, system vs. code
- 4 The EKS substrate** — RDS, S3, ECR, identity — the AWS wiring
- 5 The run & execution model** — how a schedule becomes a pod, and the cleanup trap
- 6 Single sign-on with Keycloak** — auth at the edge with oauth2-proxy
- 7 Version discipline** — the coupling rule that breaks deployments
- 8 Step-by-step deployment** — the runbook, in order
- 9 Gotchas & pre-flight checklist** — what bites juniors first

1 · The big picture

We are taking a Dagster deployment that runs on a laptop and moving it onto Amazon EKS so it can run securely, survive restarts, scale on demand, and be used by a whole team behind company login.

Before any detail, hold this one picture in your head. A user opens the Dagster UI in a browser. The request lands on a cluster, but first passes through a single-sign-on gate. Inside the cluster, two small "system" services run all the time, and each data domain runs as its own isolated service. Heavy work happens in short-lived pods that appear, do their job, and disappear. Everything durable — the database, the data files, the container images, the secrets — lives in AWS managed services outside the cluster, so the cluster itself stays disposable.

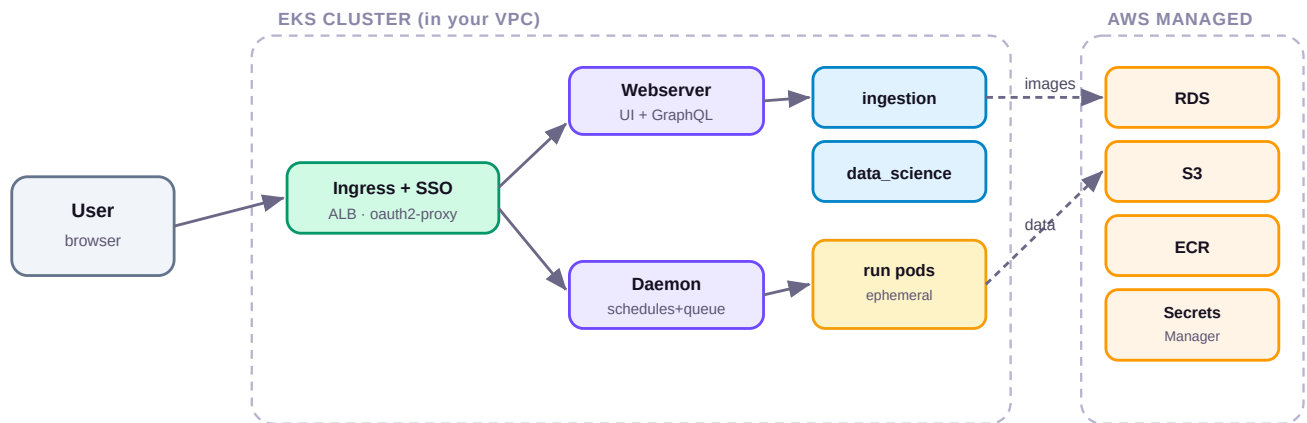


Figure 1 — The whole system on one page. Read it left to right: user → SSO → cluster → AWS managed services.

The rest of this guide unpacks each box in that figure: what it is, why it is there, and how to deploy it. If you ever feel lost, come back to Figure 1.

MENTAL MODEL

The cluster is **cattle, not pets**. Nothing important is stored inside it. If the whole cluster vanished, you could rebuild it and reconnect to the same database, files, and images. That principle drives almost every decision below.

2 · Foundations

Three technologies, in plain language. If you already know them, skip to Section 3.

Kubernetes and EKS

Kubernetes (often "K8s") is software that runs and supervises containers across a fleet of machines. You describe the desired state — "I want two copies of this service running" — and Kubernetes makes reality match, restarting things that crash and rescheduling them when a machine dies. A running container is wrapped in a **Pod** (the smallest unit K8s manages). A **Deployment** keeps a set of identical pods alive. A **Service** gives those pods a stable internal address. A **Job** runs a pod until a task completes, then stops.

EKS (Elastic Kubernetes Service) is AWS running the Kubernetes control plane for you, so you don't babysit the cluster's brain. You still bring the worker machines (nodes) where your pods run. **Karpenter** is an add-on that adds and removes nodes automatically based on demand — important later, because our heavy work is bursty.

Helm

Writing Kubernetes manifests by hand is tedious and error-prone. **Helm** is a package manager: a **chart** is a bundle of manifest *templates*, and a **values.yaml** file fills in the blanks (which image, how much memory, which database). Helm combines them into final manifests and applies them in one command. You install or upgrade a whole application as a single named **release**.

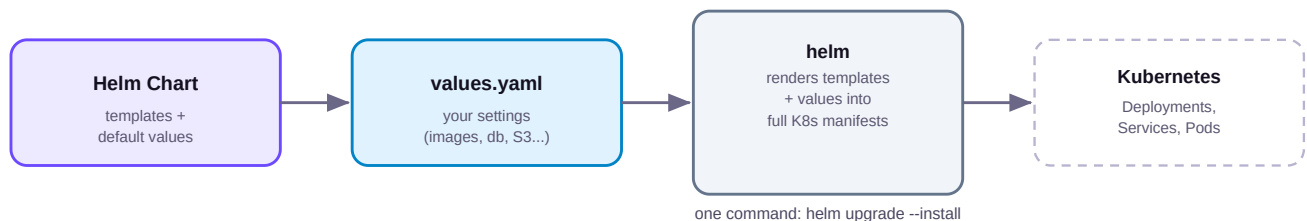


Figure 2 — Helm = templates + your values, rendered into real Kubernetes objects by one command.

Dagster

Dagster is a data orchestrator: it runs your data pipelines, tracks the **assets** they produce (tables, files, models), and shows their dependencies and run history in a UI. You write Python that declares assets and their upstream dependencies; Dagster figures out the order and the schedule. The key idea for deployment: your pipeline code is packaged as one or more **code locations** (also called workspaces), and Dagster's system services load that code and launch runs from it.

WHY THIS MATTERS

Dagster cleanly separates **system** (the always-on UI and scheduler) from **your code** (the pipelines). That separation is the single most important thing to understand before deploying — it is why there are two Helm charts, covered next.

3 · Dagster's architecture on Kubernetes

Dagster ships **two** Helm charts. Knowing which piece lives where removes most of the confusion juniors hit.

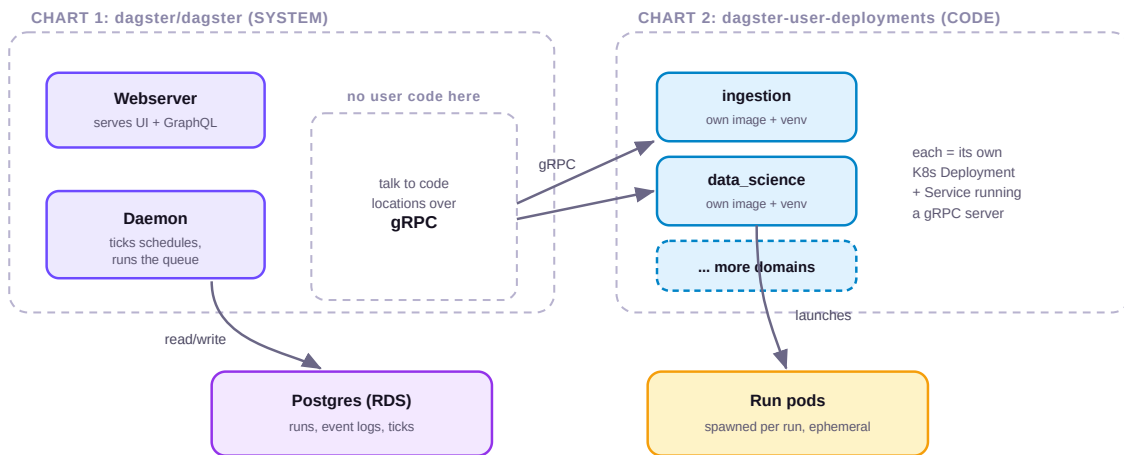


Figure 3 — Two charts. System services hold no pipeline code; they reach each code location over gRPC.

Chart 1 — the system (dagster/dagster)

This chart deploys two long-running services:

- **Webserver** — serves the UI and the GraphQL API the UI talks to.
- **Daemon** — the background engine: it ticks schedules and sensors, and works through the run queue, deciding what to launch and when.

Crucially, **neither contains your pipeline code**. They discover and talk to your code over a network protocol called **gRPC**. The chart can also deploy a Postgres pod for convenience — we will turn that off and use a managed database instead (Section 4).

Chart 2 — the code (dagster/dagster-user-deployments)

Each data domain — `ingestion`, `data_science`, and so on — becomes one entry here. For each entry, the chart creates a Kubernetes Deployment and Service that runs a small gRPC server hosting that domain's code. Each runs in **its own image with its own Python environment**, fully isolated from the others. A crash or a heavy dependency in one domain cannot affect another, and you can redeploy one domain without touching the rest.

WHY TWO SEPARATE CHARTS (TWO RELEASES)

Running system and code as separate Helm releases means you can ship a new version of a pipeline many times a day without ever restarting the UI or scheduler. The platform team owns the system release; each domain team owns its own code release. This is the recommended production pattern.

Where state lives

Two durable stores sit beneath everything. **Postgres** holds all metadata — every run, every event log line, every schedule tick. **Run pods** are spawned per run to do the actual work and then disappear. The next two sections are about doing both of these *correctly on AWS*, which the vanilla tutorial does not.

4 · The EKS substrate (the AWS wiring)

The basic Helm tutorial uses an in-cluster database and pasted-in AWS keys. Both are fine for a demo and wrong for production. Here is what a real EKS deployment uses instead.

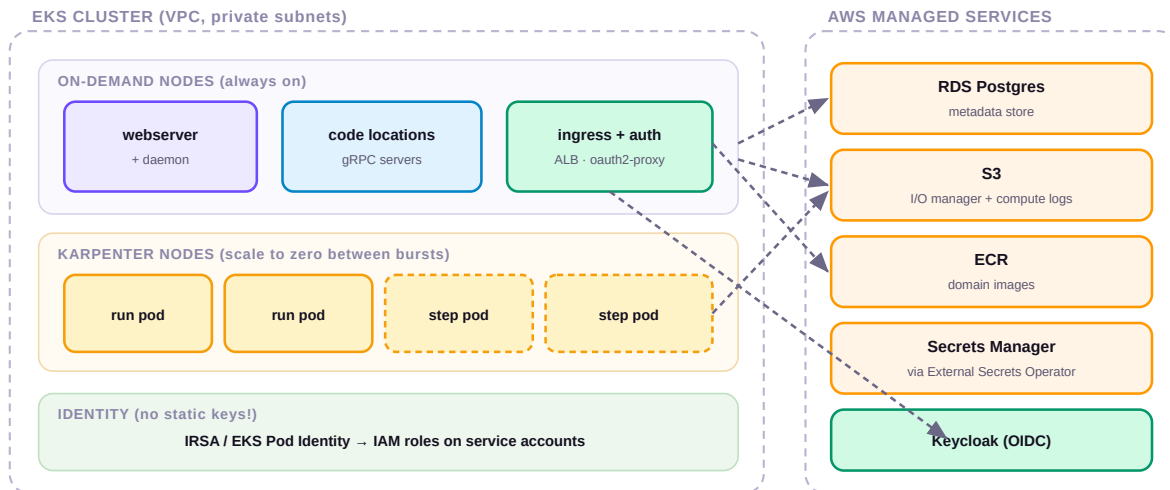


Figure 4 — Durable state and identity live in AWS managed services; the cluster stays disposable.

Postgres → Amazon RDS

Do not use the Postgres pod the chart can create — a database inside the cluster is not safe for production data. Point Dagster at a managed **RDS** (or Aurora) instance and set `postgresql.enabled=false` in the system chart. RDS gives you backups, failover, and the performance headroom you will need as metadata grows.

Object storage → Amazon S3

Dagster's **I/O manager** reads and writes asset values to **S3**, and compute logs also go to S3. This is essential in a multi-domain setup: if `data_science` consumes an asset produced by `ingestion`, that value passes through shared storage, because the two run in separate containers that cannot see each other's local disk.

Identity → IRSA or EKS Pod Identity (never static keys)

The tutorial pastes an `AWS_ACCESS_KEY_ID` into config. Don't. On EKS, attach an IAM role to a Kubernetes **service account** via **IRSA** or the newer **EKS Pod Identity** (prefer Pod Identity on a new cluster). Pods then receive AWS permissions automatically, with no long-lived secret to leak.

THE #1 EKS-DAGSTER FOOTGUN

The S3 permission must be granted to **three** things: each code-location deployment, **and the run/step pods they launch**. The I/O manager reads and writes from inside the ephemeral run worker — not the gRPC server. Forget the run-pod role and your assets work in local dev but fail with `AccessDenied` in the cluster. This is the most common first-deploy failure.

Images → ECR · Secrets → Secrets Manager

Build one image per domain, push to **ECR**, and reference by immutable tag or digest (pull access must also reach the run pods). Store secrets — database password, SSO client secret — in **AWS Secrets Manager** and sync them into the cluster with the **External Secrets Operator**, so nothing sensitive lives in `values.yaml` or Git.

5 · The run & execution model

This is where Dagster meets Kubernetes most directly — and where an innocent setting quietly creates an operational problem at scale.

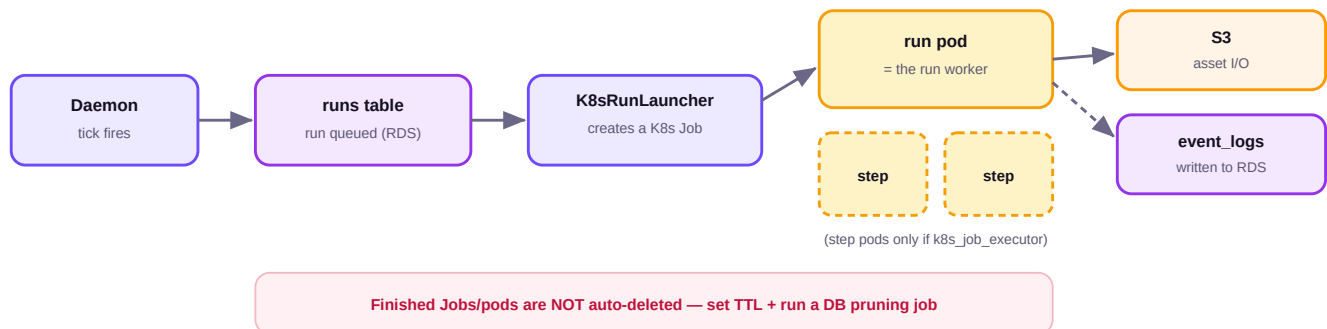


Figure 5 — From schedule tick to finished run. The red bar is the trap most teams hit.

How a run happens

1. The **daemon** ticks a schedule and writes a queued run into the Postgres **runs** table.
2. The **K8sRunLauncher** (the default) creates a Kubernetes **Job** for that run.
3. That Job is the **run worker** pod. It uses the *same image* as the code location it came from, so the pipeline code and dependencies are present.
4. If you use the **k8s_job_executor**, each step of the run launches in its own additional pod. With the default in-process executor, the whole run executes inside that single run pod.
5. Asset values flow to/from **S3**; every event is written to **Postgres**, which is what the UI later displays.

THE CLEANUP TRAP

Finished run and step Jobs are **not** deleted automatically — Kubernetes keeps them so you can inspect results. At a 5-minute schedule across several domains that is well over a thousand runs a day, each leaving objects behind. Two fixes are mandatory: (1) set **ttlSecondsAfterFinished** so Kubernetes garbage-collects finished Jobs, and (2) run a Dagster **pruning job** against Postgres — the built-in retention only trims schedule ticks, not the **runs** and **event_logs** tables, which otherwise grow without limit.

Choosing an executor (a real decision)

Do **not** reach for **k8s_job_executor** everywhere. For light, frequent pipelines (e.g. a 5-minute metrics job), pod-per-step startup latency rivals the run interval and multiplies churn — use the in-process executor so each run is a single pod. Reserve **k8s_job_executor** for heavy work like model training, where isolating steps and giving each its own resources actually pays off. Executors can differ per job; mix them deliberately.

NODE STRATEGY

Put ephemeral run pods on **Karpenter**-managed nodes that scale to zero between bursts, and keep the always-on webserver and daemon on stable on-demand nodes. Bursty, schedule-driven work is exactly what Karpenter is good at.

6 · Single sign-on with Keycloak

Open-source Dagster has **no built-in login** — anyone who can reach the URL has full access. We fix that by enforcing authentication at the edge, in front of the webserver, using the company's existing Keycloak identity provider.

The pattern: authenticate at the edge

Requests enter through an **ALB ingress** and hit **oauth2-proxy** before reaching Dagster. oauth2-proxy speaks **OIDC** (OpenID Connect) to **Keycloak**. If the user has no valid session, they are bounced to Keycloak to log in (with whatever the company enforces — MFA, device checks). Once authenticated, oauth2-proxy forwards the request to the webserver and adds identity headers describing the user. Dagster itself never handles a password.

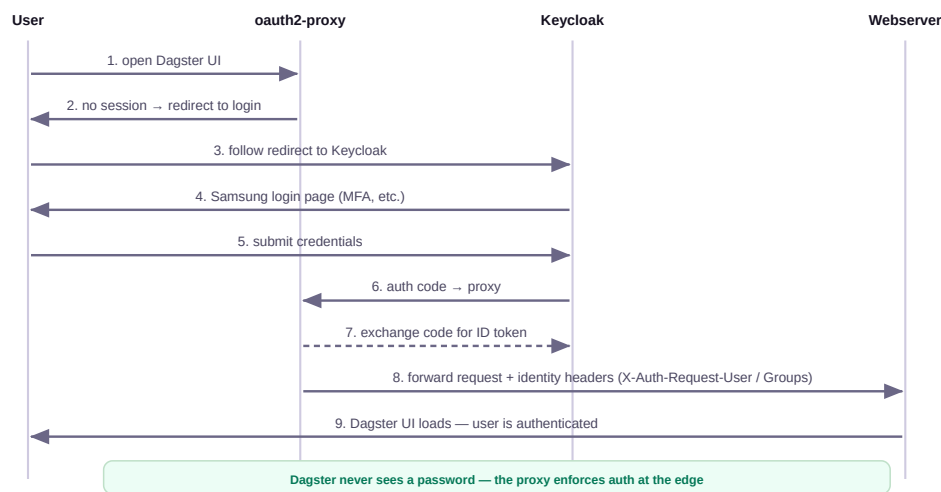


Figure 6 — The OIDC login handshake. Steps 1–9 happen in a fraction of a second after first login.

Two stages, on purpose

Stage	What it gives you	Limitation
Interim — dagster-authkit	A login page and simple role-based access (admin / editor / launcher / viewer) wrapping the webserver. Fast to stand up; good for local and early testing.	Roles are instance-global — it cannot say "team A may run only the ingestion domain."
Target — Keycloak + oauth2-proxy	Real enterprise SSO through the company identity provider, MFA included, sessions managed centrally. The production answer on EKS.	Requires a Keycloak client registration from the identity team before it can be wired.

WHY EDGE AUTH BEATS IN-APP AUTH HERE

Dagster's UI talks to a single GraphQL endpoint, so per-action rules are awkward to enforce inside the app. Terminating auth at the ingress keeps Dagster simple and puts identity in one audited place, reusing infrastructure the company already runs.

PER-DOMAIN ACCESS

Real per-domain isolation (team A → ingestion only) comes from group claims in the Keycloak token, evaluated at the proxy — not from authkit. Plan for it in the OIDC stage.

7 · Version discipline

One rule prevents a whole category of confusing failures: keep every Dagster version in lockstep.

The Helm chart is versioned with the same numbers as the Dagster Python library. So the chart version, the system images (webserver, daemon), and **every code-location image** must all be the *same* Dagster version. A mismatch shows up as a gRPC handshake failure when the webserver tries to load a code location — a baffling error if you don't know the cause.

The two version numbers

Dagster uses two version tracks that move together and must **both** be pinned:

Track	Packages	Example
Core	<code>dagster</code> , <code>dagster-webserver</code> , <code>dagster-graphql</code>	<code>1.13.x</code>
Integration libraries	<code>dagster-postgres</code> , <code>dagster-aws</code> , <code>dagster-k8s</code> ...	<code>0.29.x</code>

They map one-to-one: core `1.13.10` goes with libraries `0.29.10` . There is **no** `dagster-postgres==1.13.10` — installing a library with the core version string simply fails. Keep both numbers in one source of truth (a `versions.toml`) and feed them into every image build, so they can never drift apart.

COMMON MISTAKE

Bumping the chart (`helm upgrade --version 1.13.11`) but leaving code-location images on an older version. Always move the chart and all images together, and re-run your tests after a bump.

8 · Step-by-step deployment

The order matters. Each step assumes the previous one succeeded. Adapt names and ARNs to your environment.

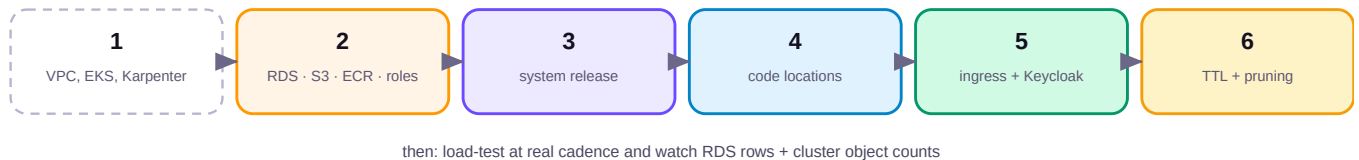


Figure 7 — Deploy in this order. Never wire auth or run a load test before the core loop is green.

Step 1 — Cluster foundation

Stand up the VPC and EKS cluster, install Karpenter, and create the namespaces. Confirm `kubectl get nodes` shows healthy nodes before continuing.

Step 2 — AWS resources and identity

Create the RDS instance, the S3 bucket(s), and the ECR repositories. Create the IAM roles and bind them to the service accounts (IRSA / Pod Identity) — including the role used by run pods. Install the External Secrets Operator and point it at Secrets Manager.

Step 3 — Add the chart repo and install the system release

```
# register Dagster's Helm chart repository
helm repo add dagster https://dagster-io.github.io/helm
helm repo update

# install the system release, pinned, Postgres disabled, pointed at RDS
helm upgrade --install dagster-system dagster/dagster \
  --namespace dagster --create-namespace \
  --version 1.13.x \
  -f values-system.yaml
```

In `values-system.yaml` you set `postgresql.enabled=false`, the RDS connection (from a synced secret), the run launcher and retention settings, and the `workspace` list pointing at the code-location services from Step 4.

Step 4 — Install the code-location release

```
# a separate release so domains deploy independently of the system
helm upgrade --install dagster-code dagster/dagster-user-deployments \
  --namespace dagster \
  --version 1.13.x \
  -f values-code.yaml
```

`values-code.yaml` lists one deployment per domain — image (from ECR), the `dagsterApiGrpcArgs` pointing at its module, the service account with S3 access, and env from synced secrets. Open the UI (via port-forward for now) and confirm every code location loads **green** and a test asset materializes.

Step 5 — Ingress and SSO

Deploy the ALB ingress and oauth2-proxy, configured against the Keycloak client your identity team registered. Verify that hitting the URL now redirects to Keycloak and, after login, returns you to a working UI.

Step 6 — Cleanup and validation

Enable `ttlSecondsAfterFinished` on launched jobs and schedule the Postgres pruning job. Then load-test at the real schedule cadence and watch two numbers: RDS row growth and Kubernetes finished-object counts. If both stay flat over a few hours, the deployment is healthy.

9 · Gotchas & pre-flight checklist

The failures juniors hit first, and a checklist to catch them before they happen.

Top gotchas

Symptom	Likely cause
Assets work locally, <code>AccessDenied</code> in cluster	S3 IAM role not bound to the run pods (only to the code location). See Section 4.
Code location won't load; gRPC handshake error	Version mismatch between chart / system image / code image. See Section 7.
Cluster slows down, <code>kubectl</code> lags after days	Finished Jobs never cleaned up — missing TTL. See Section 5.
RDS storage climbs steadily, UI gets slow	No DB pruning — <code>runs</code> / <code>event_logs</code> growing unbounded. See Section 5.
Pipelines lag behind a 5-minute schedule	Using <code>k8s_job_executor</code> for light work — pod startup overhead. See Section 5.
Anyone with the link sees everything	No edge auth wired yet — oauth2-proxy/Keycloak not in front. See Section 6.

Pre-flight checklist

#	Check
1	RDS reachable; <code>postgresql.enabled=false</code> in the system values.
2	S3 bucket exists; IAM role bound to code locations and run pods.
3	ECR images pushed; pull access reaches run pods.
4	Secrets synced from Secrets Manager via External Secrets Operator.
5	Chart version == system image version == every code image version (core + lib).
6	Every code location loads green; a test asset materializes end-to-end.
7	Schedules tick on the daemon at the agreed cadence.
8	Executor chosen per job (in-process for light, <code>k8s_job</code> for heavy).
9	TTL set on launched jobs; Postgres pruning job scheduled.
10	Ingress redirects to Keycloak; login returns to a working UI.
11	Karpenter scales run nodes up under load and back to zero after.
12	Load test at real cadence: RDS rows and cluster objects stay flat.

IF YOU REMEMBER ONLY THREE THINGS

(1) The cluster is disposable — durable state is in RDS and S3. (2) Grant S3 to the run pods, not just the code locations. (3) Pin one Dagster version everywhere, and clean up finished jobs and old DB rows from day one.